作品编号：CACR2026EA16XZ

作品类别： □软件设计 □硬件制作 ■工程实践

□密码应用技术 □其它

2026 年第十一届全国密码技术竞赛作品设计报告

作品题目：ML-KEM 解封装实现的时间侧信道检测与常数时间防护

（暂定题名）

2026 年 10 月 8 日

中国密码学会

基本信息表

编号：CACR2026EA16XZ

作品题目：ML-KEM 解封装实现的时间侧信道检测与常数时间防护（暂定题名）

作品类别：□软件设计　□硬件制作　■工程实践　□密码技术应用　□其它

作品内容摘要：

本作品面向 ML-KEM 解封装实现的本地审计需求，设计并实现 C11 审计与精确复算系统。系统采用固定 ML-KEM-768 便携实现，以独立公开向量及接口检查验证功能，在完整解封装窗口中记录逐次时间，并关联保存源码、产物、环境和成本。针对结果难以核查的问题，将数据资格与统计数值分开输出，实现有界成对数据的整数精确复算。测试中，八个固定任务与 SciPy 尾计数及判定一致，预导入对照后的耗时比值中位数为 0.001562070。同语言对照六个非零任务的增量/完整求和中位比为 0.261640 至 0.319174，限于本次 macOS/ARM 构建；40964 份有限模型及 24576 份后赋标签结果均通过独立核对。同机人工机制复测中，掩码形态在五个预定种子的原名义检验里均未再检出差异。作品已完成封存包新目录复算，支持本地实现审查及离线复核；统计结果均按对应模型解释，不外推为全库恒时保证。

关键词（五个）：ML-KEM；时间侧信道；常数时间；证据资格；可复现性

目 录

第 1 章 研究背景与作品目标.....1
1.1 后量子密码实现的工程审计需求.....1
1.2 现有实现与统计工具的能力边界.....1
1.3 作品目标与总体技术路线.....3
第 2 章 ML-KEM 实现审计的理论基础.....4
2.1 ML-KEM-768 的接口与功能正确性.....4
2.2 外部输入检查与隐式拒绝语义.....5
2.3 执行时间的影响因素与样本单位.....6
2.4 成对随机化与整数精确检验.....6
2.5 运行分位数与相对性能开销.....7
第 3 章 本地审计与精确复算系统设计.....9
3.1 系统架构与模块协同.....9
3.2 固定快照管理与多实现构建隔离.....10
3.3 普通输入组织与完整解封装计时.....11
3.4 运行记录组织与来源一致性校验.....11
3.5 数据资格控制与精确复算实现.....13
3.6 材料复现流程与离线结果展示.....13
第 4 章 实现安全性与审计结论的适用范围.....15
4.1 本地审计的对象与可见信息.....15
4.2 标准语义与外部接口保护.....16
4.3 恒时实现检查与历史案例解释.....16
4.4 统计资格与条件错误控制.....17
4.5 复现一致性与研究结论的区分.....18

目　录（续）

第 5 章　系统测试与实验结果分析.....19
5.1　原生测试环境与分层验证方案.....19
5.2　独立向量验收与工程边界检查.....20
5.3　普通解封装性能与跨运行波动.....20
5.4　精确复算效率与成本边界复核.....23
5.5　有限模型下的误拒绝与检出表现.....25
5.6　真实参考时间的后赋标签控制.....27
5.7　历史案例审计与封存材料复算.....28
5.8　同机人工机制的防护前后复测.....29
第 6 章　作品特色与应用前景.....31
6.1　面向固定任务的专用复算效率.....31
6.2　数据资格与解释范围的一体化设计.....31
6.3　本地审计与密码工程教学.....32
第 7 章　总结与展望.....33
7.1　工作总结.....33
7.2　现有限制与后续工作.....33
参考文献.....34
附录 A　实验材料与结论对应关系.....35
附录 B　普通性能的完整分位数.....38

第 1 章　研究背景与作品目标

1.1　后量子密码实现的工程审计需求

后量子密码算法的标准化，使密码软件的关注点从算法选择延伸到实现质量。NIST 于 2024 年发布 FIPS 203，规定了基于模格的密钥封装机制 ML-KEM。[1] 标准在具体系统中的落实，还取决于输入检查、随机源处理、编译优化及错误返回等实现细节。

解封装接收外部密文，使用本地解封装密钥完成计算，既要保持有效输入的功能，又要遵循无效密文的隐式拒绝语义。一次封装与解封装的结果一致，不能覆盖全部接口边界；若干次执行时间出现差异，也不足以区分代码形态、编译条件和环境因素。因此，**功能正确性、执行时间与结论适用条件需要在同一套审计材料中相互对应。**

这些材料往往分散保存：源码说明设计意图，统计报告描述观测差异，性能表汇总运行时间。但它们若缺少共同身份和原始记录，复核人员就难以把结论追溯到对应程序，启动成本、普通长尾及失败运行也容易被遗漏。

本作品围绕 ML-KEM-768 便携 C 实现，设计本地审计与结果复算系统。统一入口组织功能验收、普通记录、数据校验、条件分析及离线展示，将原始数据与源码、产物、环境和成本关联保存，为实现审查提供可检查、可重建的工程依据。

1.2　现有实现与统计工具的能力边界

ML-KEM 标准、密码库实现和统计分析工具分别解决不同层面的问题。FIPS 203 给出算法与接口要求；mlkem-native 提供具体实现，并明确其形式验证的范围、假设与风险。[1][2] 使用已有密码库，可以减少重新实现核心算法带来的错误，但不能省略对配置、封装层和实际编译产物的检查。一个库中存在经过证明的组件，也不意味着本作品的全部 C 程序及所有编译条件已经获得恒时证明。

统计时间分析工具从测量结果出发，寻找不同输入条件下的时间差异。dudect 采用 Welch 检验相关流程；RTLF 讨论有界第一类错误下的统计时间分析，并对理论适用条件作出规定。[3][4] 这些工作说明，检验统计量之外，数据产生方式及概率假设同样重要。将一组重复测量直接当作相互独立的样本，可能使数值结果超出其合理解释范围。

本作品采用的成对交换与完整枚举属于已有随机化方法。[5] 因而，作品不以“重新提出精确检验”作为创新，而是将固定规模整数复算、输入资格控制以及材料复现结合起来。对照使用与候选相同的统计量、交换空间和尾部定义，在固定任务中比较复算成本；对于没有得到验证的适用条件，则单独输出其限制。

Breaking Bad 系统研究编译优化破坏源码恒时防御的情况，说明防护写法需与工具链和实际产物一起检查。[11] RTLF 正式勘误则更正了一项 NSS 时间差异的可利用性判断。[4] 因此，源码机制、产物形态、观测差异及可利用性分别报告；其他库的发现不构成本作品固定 ML-KEM 产物的漏洞证据。

近年来，SILENT 将依赖结构与预设容许阈值纳入时间分析，但其明确的连续情形保证仍依赖严格平稳、有限范围依赖及密度条件，主要为渐近结论。[7] WOOT 2026 对恒时模型的梳理进一步提示，保证对象还需要说明接口、加载和运行条件。[8] 本作品据此区分功能、产物、条件推断与安全结论，不将一种工具的保证移交给未验证的对象。

本作品的核心研究问题是：在固定 ML-KEM 本地审计任务中，如何让统计对象、实际产物、数据资格和复算成本相互对应。主要实证改进发生在相同统计规则的计算环节；部分资格校验与来源关联为该结果提供可审查的支撑。

表 1.1　相关对象与本作品的工作定位

对象	主要解决的问题	本作品的使用方式
FIPS 203	ML-KEM 标准定义	固定 ML-KEM-768 功能与接口口径
mlkem-native	密码算法实现及部分验证	固定快照、便携 C 配置和原始许可证
dudect / RTLF	统计时间分析	查明方法与假设，不设置失配对照
SciPy 精确交换	通用随机化计算	同一任务、同一计数定义下的强基线
本作品	本地审计和结果复算	关联输入、程序、记录、条件与结论

1.3　作品目标与总体技术路线

本作品以“功能可验、记录可查、结论可复算”为目标。系统先以独立向量和边界测试核对功能，在完整解封装调用周围建立计时窗口，再根据数据资格开展精确统计复算，并通过独立计数与对照工具核验结果。

作品形成三类材料：真实 Linux 普通功能与性能记录、人工机制案例的历史审计及同机复测、无秘密固定任务的统计与效率验证。三者分别回答软件行为、机制观察及计算正确性问题，不将不同平台或用途的记录拼接为同一次实验。

八个固定确认任务中，C11 与 SciPy 精确尾计数及判定全部一致；预导入复核后，耗时比值中位数为 0.001562070。同机人工机制复测中，掩码形态在五个种子的原检验里均未再检出差异。两项成果分别支持固定任务复算效率与特定机制的防护观察，不外推为全库安全保证。

第 2 章　ML-KEM 实现审计的理论基础

2.1　ML-KEM-768 的接口与功能正确性

密钥封装机制由密钥生成、封装和解封装三个算法组成。设封装密钥为 ek，解封装密钥为 dk，共享密钥为 K，密文为 c，其接口关系可写为

$$(ek, dk) \leftarrow KeyGen, (K, c) \leftarrow Encaps(ek). \tag{2-1}$$

接收方使用解封装密钥处理密文，得到对应的共享密钥：

$$K' \leftarrow Decaps(dk, c). \tag{2-2}$$

对于按标准生成的有效密文，正确性要求输出具有标准规定的一致性。功能测试据此核对封装和解封装的结果；独立已知答案测试进一步核对具体字节值，避免仅由被测实现自行生成预期答案。图 2.1 给出了三个算法的功能关系。

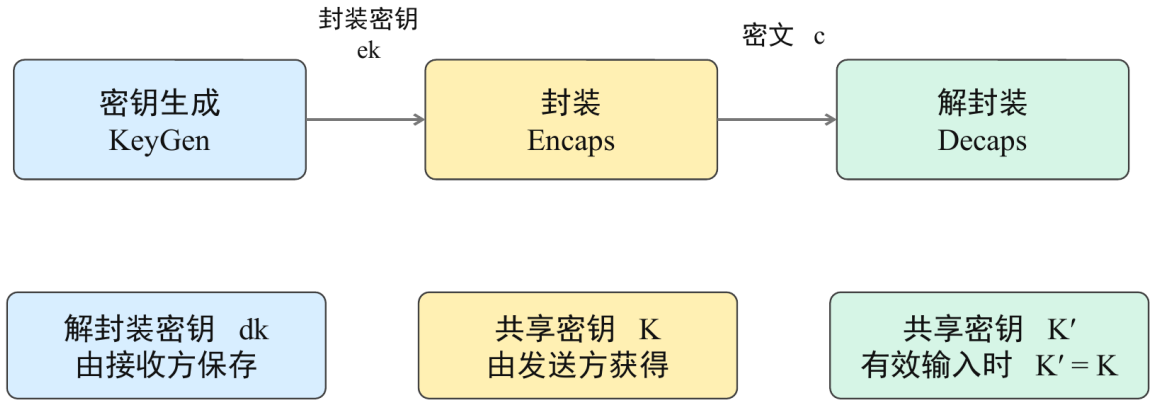


图 2.1　密钥封装机制的功能关系

表 2.1　ML-KEM-768 的接口对象长度

接口对象	本文记号	长度 / 字节
封装密钥（公钥）	ek	1184
解封装密钥（私钥）	dk	2400
密文	c	1088
共享密钥	K	32

本作品使用固定快照的便携 C 后端。版本号、标准口径和构建配置共同限定了测试对象。旧 Kyber 的研究材料与最终 ML-KEM 标准分别引用，避免将算法演进前的结论直接套用于当前实现。通过已知答案测试也只说明相应测试范围内的功能符合性，不等同于 FIPS 密码模块认证。

2.2　外部输入检查与隐式拒绝语义

外部接口与密码内核承担不同职责。接口首先检查指针和精确长度；只有满足内核输入约束的缓冲区才进入计算。对于长度正确的无效密文，程序继续保留标准规定的隐式拒绝语义，且公开错误状态不额外暴露内部验证结果。图 2.2 说明了这两类边界的区别。

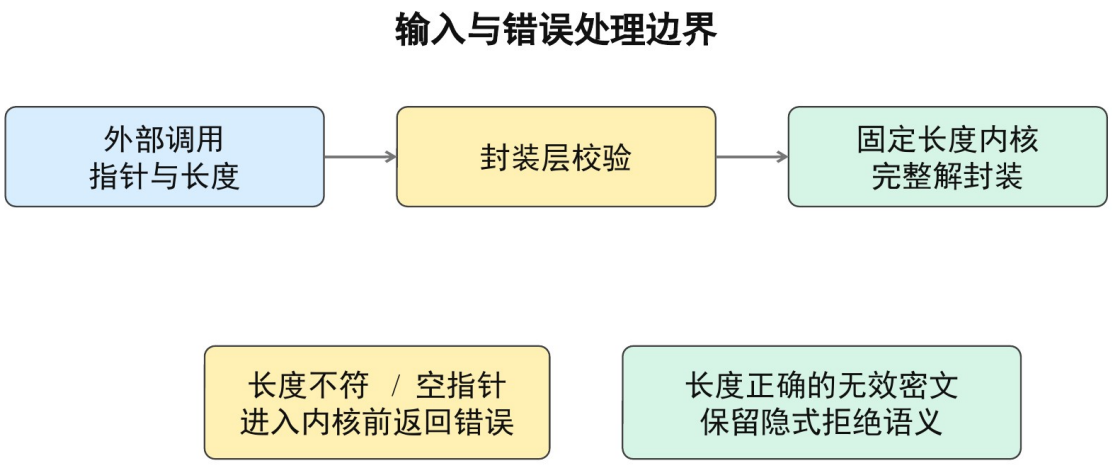


图 2.2　封装层输入检查与内核处理边界

对接口而言，接收长度为 1088 字节的输入，只表示它具备进入解封装处理的结构条件，并不表示密文有效。普通记录中的类别也仅表示生成规则：有效封装、单比特修改以及随机字节输入。程序不通过读取内部接受标志为样本重新标注，以免把本应封闭的验证信息引入外部分析。

系统随机源用于普通运行，失败必须向调用方传播；公开种子的确定性入口仅服务于明确的测试材料。两者在用途上分开，使复现能力不会改变业务密钥生成所要求的随机性。公开日志只保存运行身份和测量结果，不保存真实私钥及共享密钥。

2.3　执行时间的影响因素与样本单位

设第 j 次完整解封装调用前后的单调时钟读数分别为 t_j^{start} 和 t_j^{end}，保存的时间差为

$$x_j = t_j^{end} - t_j^{start}. \tag{2-3}$$

该时间差由多个因素共同影响。为说明测量的解释范围，可将其概念性地表示为

$$x_j = \tau_j^{impl} + \eta_j^{env} + \eta_j^{clock}. \tag{2-4}$$

其中，τ_j^{impl} 表示实现相关耗时，另外两项概括运行环境和读钟带来的影响。式（2-4）是解释测量来源的示意分解，不是已经识别各分量的拟合模型，也不预设噪声服从正态分布或彼此独立。观察到总时间不同，仍需进一步核对影响因素，才能讨论差异来源。

本作品保留逐次值，同时将一次运行及成对运行作为独立的组织层次。单次调用、运行块和确认任务分别有各自分母：54000 条调用记录不能直接写成 54000 个独立确认任务，八个固定复算任务也不能因为每个重复三次而变成二十四个不同任务。这一划分决定了性能汇总和统计解释的边界。

2.4　成对随机化与整数精确检验

对于已经满足成对分析要求的数据，每对两侧须具有相同调用次数和时间单位，差值及累计值须满足整数范围约束。设第 k 对的两个总耗时为 A_k、B_k，成对差及其观测统计量定义为

$$d_k = A_k - B_k, T_{obs} = \left| \sum_{k=1}^{m} d_k \right|. \tag{2-5}$$

对内交换两个标签等价于改变该对差值的符号。记所有交换状态构成集合 $\Omega = \{-1, +1\}^m$，则每个状态 ε 对应的统计量为

$$T(\varepsilon) = \left| \sum_{k=1}^{m} \varepsilon_k d_k \right|, |\Omega| = 2^m. \tag{2-6}$$

完整枚举后，采用包含等值的上尾计数，精确概率为

$$p_{exact} = \frac{\sum_{\varepsilon \in \Omega} 1\left[T(\varepsilon) \geq T_{obs} \right]}{2^m}. \tag{2-7}$$

该定义保留观测统计量对应的交换状态，不通过随机抽样估计分母，也不对等值边界作浮点近似。本作品将成对单位数限制在 4 至 20 之间，并使用固定阈值 $\alpha=0.01$ 在这个范围内，最大交换空间为 $2^{20}=1048576$，能够完整计数。图 2.3 给出了从成对数据到交换状态的关系。

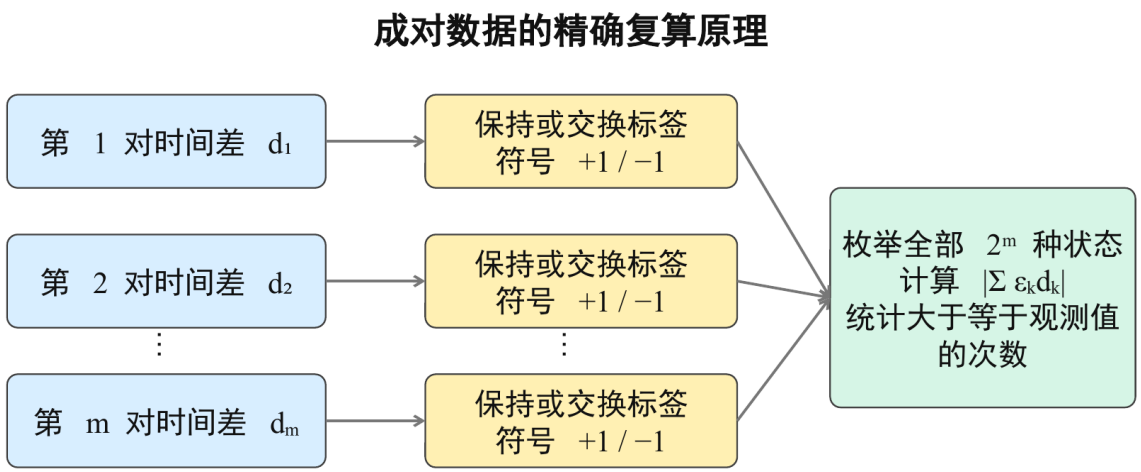


图 2.3　无秘密成对数据的精确复算原理

精确枚举消除的是给定交换空间中的抽样误差，并不会自动消除数据产生过程的偏差。把式（2-7）解释为有效随机化检验，需要对内标签交换依据与无后效条件成立；效应区间则另有运行单位独立同分布等要求。程序可以检查声明是否齐全以及记录是否自洽，但不能凭一个声明字段证明这些条件真实成立。

完整枚举与增量更新的计算思想已有直接先验。Diaconis 与 Holmes 在 1994 年研究 Gray 编码用于随机化完整遍历及局部更新加速。[9] 本作品的贡献是将既有原理落实为固定任务的 C11 有界整数实现，核对尾计数、并列与溢出边界，并保留相同工作量下的实际成本。

2.5　运行分位数与相对性能开销

对一份包含 n 个逐次时间的运行记录，均值采用全部正式样本计算：

$$\bar{x}=\frac{1}{n}\sum_{j=1}^{n}x_j. \tag{2-8}$$

将数据从小到大排序为 $x_{(0)},\dots,x_{(n-1)}$，描述分位数采用固定零基下标，不作插值：

$$Q_p=x_{(r)},r=\lfloor p(n-1)+0.5\rfloor. \tag{2-9}$$

因此，P50、P90、P99 表示在同一次运行中不同位置的典型耗时，能够补充均值无法完整表达的尾部信息。历史输出曾使用不够准确的分位数名称，本作品保留封存数值和复算兼容性，并在方法说明中明确实际定义。

设参考实现与比较实现分别有 B 个对应运行块，平均差及相对开销定义为

$$\overline{\Delta}=\frac{1}{B}\sum_{b=1}^{B}\left(\bar{x}_{b,cmp}-\bar{x}_{b,ref}\right),R=\frac{\overline{\Delta}}{B^{-1}\sum_{b=1}^{B}\bar{x}_{b,ref}}. \tag{2-10}$$

相对开销可以为负。负值仅表示在当前记录中的平均时间更短，不意味着比较实现更安全，也不保证换一台机器仍会得到相同结果。本作品保留所有运行块，使用与采集身份对应的描述指标，并明确探索性区间的适用模型。

第 3 章　本地审计与精确复算系统设计

3.1　系统架构与模块协同

针对实现审查中对象、记录与结论容易脱节的问题，本作品采用统一入口、隔离实现和关联记录相结合的设计。核心算法封装、命令行入口、输入生成、计时、统计与运行校验均使用 C11；CMake 与 CTest 统一管理构建和检查。系统结构如图 3.1 所示。

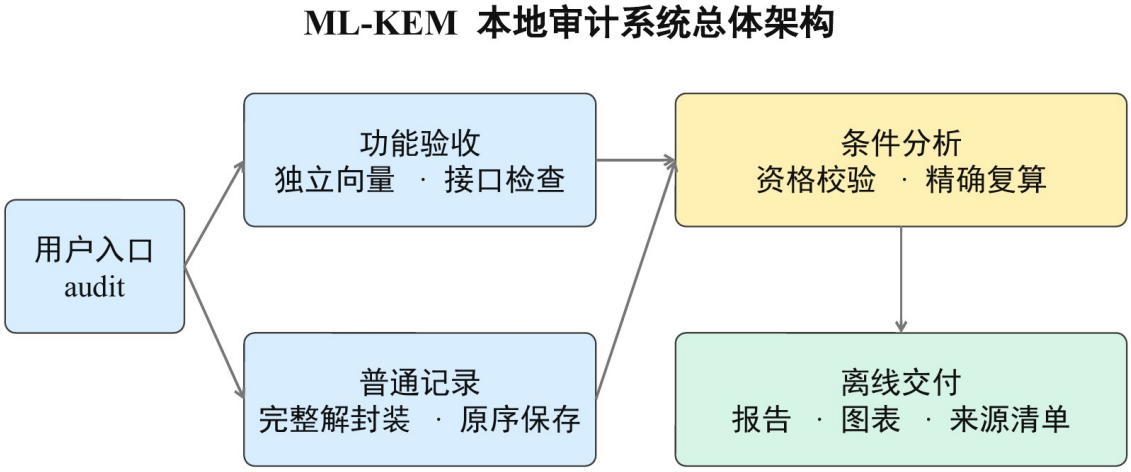


图 3.1　ML-KEM 本地审计系统总体架构

系统以 audit 为普通使用入口。功能验收首先核对独立向量、接口及随机源错误；普通记录随后保存完整解封装时间和元数据；分析模块在读取原始材料后核验结构与资格，生成描述或条件化统计结果；离线导出则将结果和来源组织为可阅读的页面。这样，使用者能够从报告返回原始记录，而不必只依赖结论文字。

历史案例使用已经归档的材料重分析。默认交付流程不将旧主动机制计时作为普通入口，避免历史材料与当前运行记录混用。所有功能围绕单机 ML-KEM-768 的审查任务组织，没有将作品扩展为通用云服务或多算法框架。

表 3.1　主要模块及其交付内容

模块	核心职责	主要输出
功能验收	独立向量、接口边界与结果等价	检查日志及通过状态
普通采集	生成普通输入并计时	逐次 CSV、元数据及成本
数据校验	核对身份、结构与完成声明	数据资格与错误原因
统计复算	描述指标及限定成对分析	计数、概率和适用范围
历史审计	重建既有规则下的输出	原报告一致性记录
离线交付	汇总报告和来源	离线页面、清单及图源

3.2　固定快照管理与多实现构建隔离

密码实现采用 mlkem-native 固定提交 b3ba7b32773e657dd37f6f87bce82528459ad8a4。上游源码和许可证保持原样，自有封装与补丁分开管理。构建配置固定为便携 C 路线，使所用后端能够通过编译输入和实际链接关系核对。只在文档中声明某种实现并不足够，实际产物也必须与该声明相符。

历史工程保留 reference、vulnerable、hardened 与 deployable 四种标签。它们分别表示参考实现、人工机制回归、保留相同函数级优化控制的掩码对照，以及去除该控制的正常优化编码对照。标签是历史产物身份，不代表四种均可直接用于业务部署。特别是 deployable 这一历史名称，不应被理解为已经通过完整安全认证。

表 3.2　实现标签及其解释范围

历史标签	主要用途	必须保留的限定
reference	固定快照参考对象	仅对应所记录的后端和编译条件
vulnerable	人工机制回归档案	不是新发现的部署漏洞
hardened	相同优化控制下的编码对照	不是完整加固交付件
deployable	正常优化下的编码形态对照	历史名称不构成部署安全结论

动态检查产物与正式采集产物分开构建、分时运行。ASan/UBSan 用于发现内存和未定义行为问题，其时间不进入正式性能表。当前固定配置不启用 LTO；实际 include 路径、值屏障和产物信息分别核验。自有构建身份记录曾出现路径模板错误，

已单独修正并保留记录，该修正不改变密码运算，项目记录见附录 A 中的固定版本适用性审查。

3.3　普通输入组织与完整解封装计时

普通采集使用 valid、bit-flipped、random 三类输入，比例为 1:1:1，密文长度均为 1088 字节。类别由生成规则决定，公开种子用于重建输入顺序。正式运行前先执行固定预热；输入生成、内存分配以及后续结果消费和文件输出均放在计时窗口之外，窗口内覆盖封装层和内核的完整解封装调用。

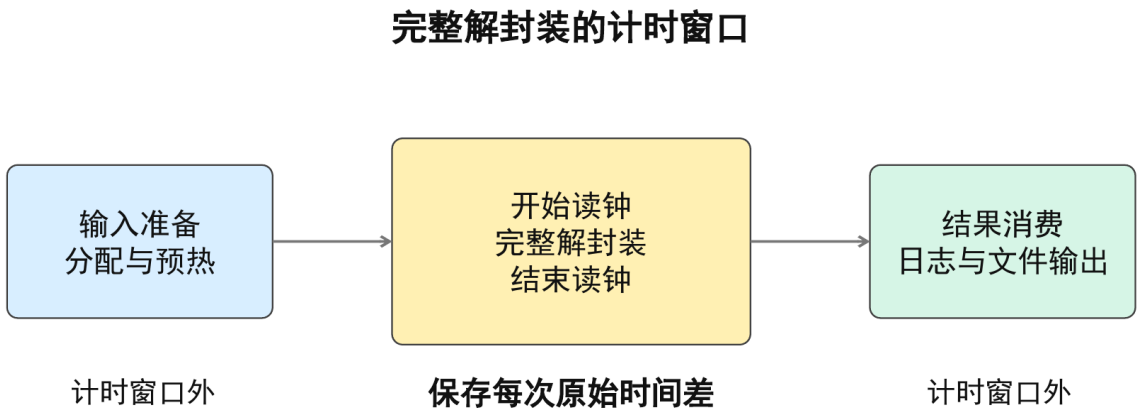


图 3.2　完整解封装的计时窗口

记录使用 CLOCK_MONOTONIC，时间单位为纳秒。纳秒是表示单位，不能代替对时钟分辨率及实际读钟行为的检查。前置参考测量中，时钟报告分辨率为 1 ns，最小正读钟增量观察值为 19 ns；每次另存 10000 个读钟增量和 1000 个空窗口值。系统不将这些开销直接从正式样本中扣除，避免未经验证的逐项校正改变原始记录。

为保留运行环境的影响，本作品保存普通长尾，不按结果大小删除观测。较大的 P99 与均值偏移都可能反映调度、缓存状态或其他系统因素。只有在明确的数据处理规则下，才对这些观测作描述性解释；“删除异常后更稳定”并不是本作品的默认做法。

3.4　运行记录组织与来源一致性校验

一次运行关联保存逐次数据、元数据和成本记录。逐次 CSV 提供运行标识、实现标签、输入类别、样本序号、时间值和单位；元数据描述源码、构建、环境、时钟

及可执行文件摘要；成本记录区分预热、正式调用和外层执行窗口。三者共同决定一份数据能否进入完整分析。

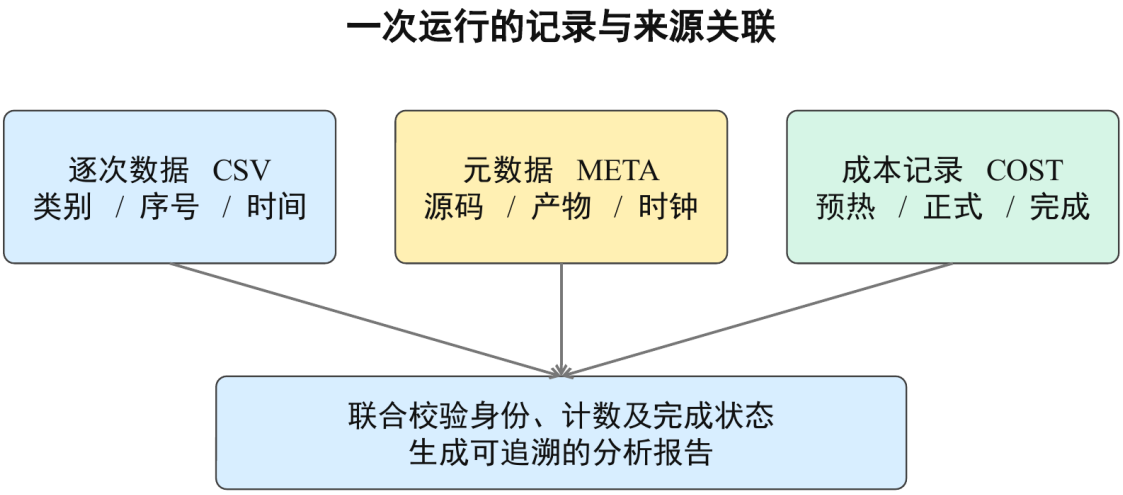


图 3.3　运行记录与来源信息的关联

表 3.3　记录字段及其检查目的

记录类别	关键内容	检查目的
逐次 CSV	运行、实现、类别、序号、时间、单位	恢复每次观测与原始顺序
元数据	源码、产物摘要、编译配置、时钟	确认实际被测对象
成本记录	预热、正式调用、父窗口、完成状态	防止遗漏成本或误认部分输出
分析计划	样本单位、交换依据、阈值、范围	约束统计解释
清单摘要	文件身份与分发对应关系	支持重建与一致性核对

软件校验包括长度、字段、计数及身份对应关系。缺少成本内容、时钟冲突、调用次数矛盾或未完成记录，不得标记为完整结果。后续校验修正复用了已有读取器，保持旧数据和统计规则不变。这种处理使“文件存在”与“材料可用于该分析”之间有明确区别。

摘要用于检查材料是否与给定清单一致，不能替代可信获取过程，也不是带有认证密钥的完整性证明。本作品因此同时保存来源、执行日志和清单身份，而不只在报告中列出一个散列值。离线 HTML 在完整写入后发布，失败时保留部分输出状态，避免把未完成页面误作正式结果。

3.5　数据资格控制与精确复算实现

分析模块先判断记录是否完整，再检查所请求分析的适用范围。对于普通数据，系统输出均值和分位数等描述结果；对于 4 至 20 个合格成对单位，才执行第 2 章定义的完整交换计数。图 3.4 给出了资格检查与结果输出的关系。

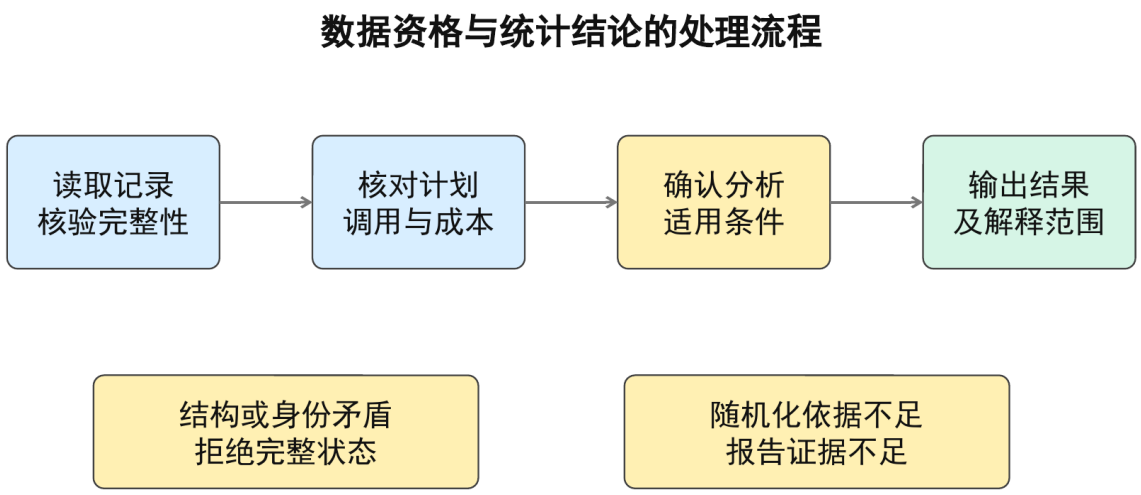


图 3.4　数据资格与统计结论的处理流程

精确复算采用整数差值与整数尾计数。其主要意义是保证与既定数学定义一致，尤其在全并列、零总和和等值上尾等边界上，不因浮点误差改变计数。实现利用相邻交换状态只改变一项符号的性质更新求和，减少固定规模枚举中的重复工作；这属于既有方法的专用工程实现，不作算法首创声明。

对于单个符号改变的状态更新，设变化位置为 r，原符号为 ε_r，原有符号和为 S，则新的和满足

$$S' = S - 2\varepsilon_r d_r. \tag{3-1}$$

式（3-1）仅描述通用整数求和关系。实现仍须检查整数范围、输入结构和枚举边界，其结果由手算用例、独立分布卷积及 SciPy 对照共同核对。提高枚举速度不能改变交换空间，也不能放宽数据的适用条件。

3.6　材料复现流程与离线结果展示

复算流程以固定材料为起点：读取源码清单，核对文件摘要，在新目录构建和验收，再读取归档数据生成结果，并与原报告逐项比较。环境、命令、退出码及结果差异共同构成执行凭证。图 3.5 展示了材料到复算输出的关系。

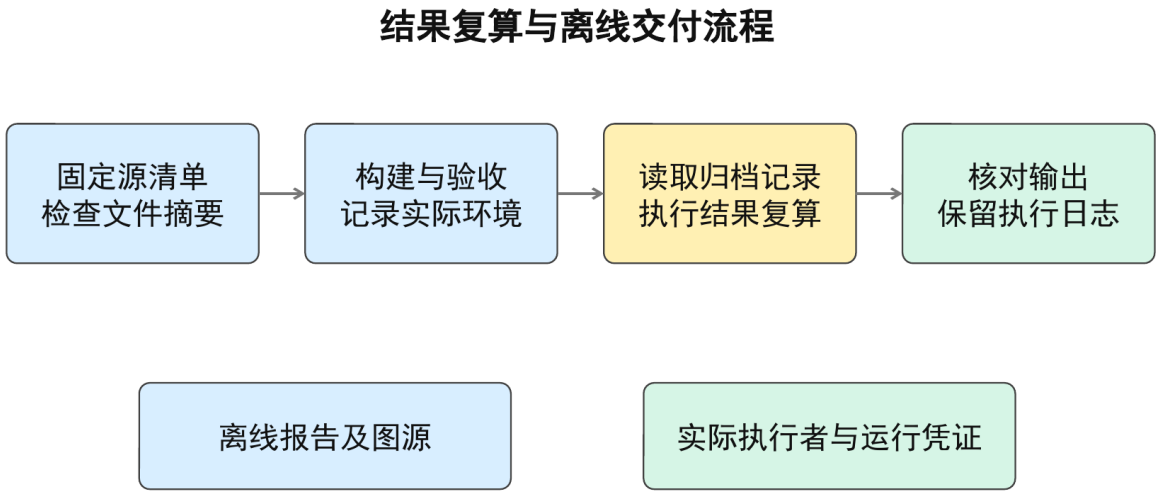


图 3.5　结果复算与离线交付流程

离线展示按证据类别组织入口，使普通记录、历史案例和无秘密统计任务能够分别查看。短演示用于说明软件流程，不能替代正式样本；文中结构图用于解释模块关系，也不充当软件实际界面截图。报告、图源和实验原始材料各自保存，便于后续调整文字与排版时核对数值来源。

2026 年 10 月 8 日，封存 v3 包已在授权 Linux 环境的新目录执行复算，26 个步骤全部退出 0。执行由授权代理完成，实际命令、环境和结果均留有记录，可供评测人员使用同一材料继续复核。

第 4 章　实现安全性与审计结论的适用范围

4.1　本地审计的对象与可见信息

本作品的审计对象是固定 ML-KEM-768 便携 C 实现及其本地封装，使用公开测试材料研究程序行为。普通流程可见外部输入类别、调用状态与完整执行时间；源码和反汇编用于解释实际产物。历史人工案例采用的白盒信息仅用于已归档材料的审查，不能推广为未知密钥条件下的观察能力。

在这一范围内，系统能够回答功能测试是否通过、记录是否完整、某种统计规则得到什么输出，以及输出可以支持到哪一层结论。作品没有取得网络攻击、密钥恢复或跨主机利用的证据，因而不作这些能力声明。图 4.1 将三类材料的对象和用途并列展示。

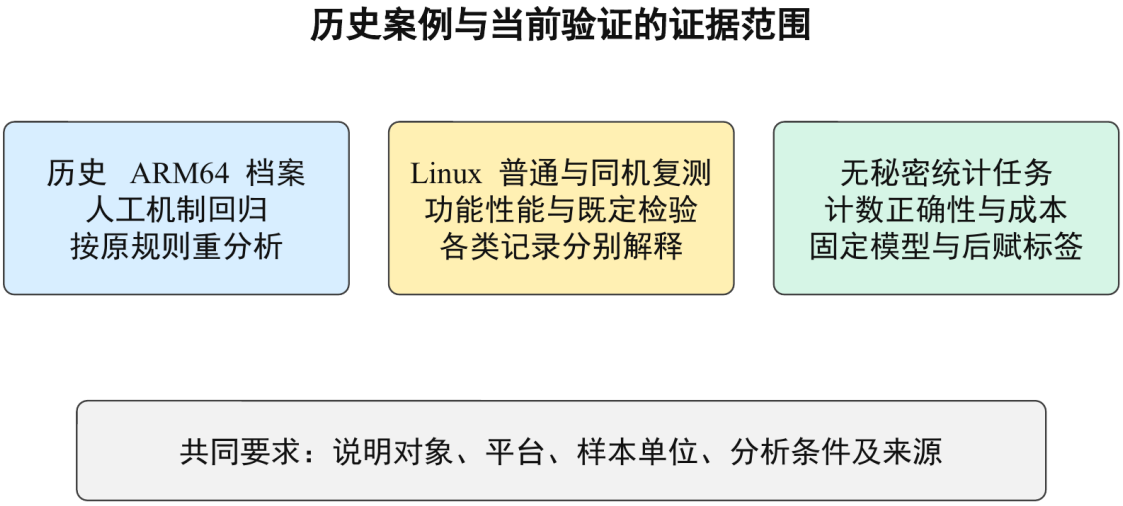


图 4.1　历史案例、普通运行与统计验证的证据范围

表 4.1　不同证据能够支持的结论

证据	可以支持	不能单独推出
独立已知答案及边界测试	已覆盖用例的功能行为	全部实现安全性或认证
源码及实际产物核对	目标路径和编译机制	真实泄露强度及可利用性
逐次普通运行记录	当前环境下的描述性能	秘密归因或防护有效性
满足条件的统计结果	对应模型下的差异判断	假设本身自动成立
固定数据逐字复算	规则与输出能够重建	原结论不再需要审查

4.2　标准语义与外部接口保护

实现审计首先要求保持密码语义。长度与空指针检查在封装层进行，防止短缓冲区到达固定长度内核；正确长度的无效密文保留隐式拒绝；普通运行使用系统安全随机源，失败时返回可处理的错误。日志、演示和错误状态不额外泄露私钥、共享密钥或内部验证结果。

独立向量测试与差分检查共同支撑功能核验。独立向量提供实现之外的预期答案；等价检查关注相同输入在不同实现标签下的结果是否一致。两种检查相互补充：前者避免错误实现之间“相互一致”，后者发现改动是否破坏了既有功能语义。它们均以已执行的测试范围为限，不能替代对全部程序路径的证明。

固定快照已包含值屏障及相关处理。公开问题是否适用于本作品，须结合版本、架构后端、配置与实际产物判断。自有修正 DEF-F01 补齐的是构建身份路径记录，实际密码运算未改变。因此，报告将其归为工程记录修正，不计作新发现的上游漏洞或防护效果。

4.3　恒时实现检查与历史案例解释

常数时间编程关注秘密数据是否影响可观察的执行行为，但源码中没有显式分支并不自动等于所有产物均恒时。编译器优化、目标指令及硬件假设都可能影响实际性质。mlkem-native 的说明也区分了已证明组件、C 层实现与相应前提。[2] 本作品据此核对固定版本的相关路径，并将未证明的范围保留为限制。

历史人工案例用于说明源码依赖、机器码和统计观察之间的关系。档案中的目标机制来自已有公开研究背景，原对象涉及旧 Kyber；本工程明确标注为人工机制回归，不将其表述为对当前上游实现的新漏洞发现。[6] 上游目标编码本已无分支，历史掩码对照不能据此改称“修复了上游漏洞”。

已有五种子档案也没有给出一致消除差异的结果。H1 主比较仅 4/5 达到原阈值；H2 中 hardened 仍有 1/5 显著残差。固定消息组重复使用少量密文，名义概率没有修正序列依赖。这些旧材料可以用于教学审查与规则复算，但不能仅据此判定当前 Linux 产物的防护表现。第 5.8 节另列新取得的同机复测，两个平台分别解释。报告保留其不利结果，而不通过更换阈值或删去运行重新组织结论。

4.4　统计资格与条件错误控制

零假设要求：给定交换轨道时，各允许的对内标签组合等概率；仅有零均值不足以保证这一性质。在独立等概率对内交换及无后效条件成立时，包含等值的完整随机化概率具有离散检验的错误控制性质。以 Q 表示这些适用条件，可将需要满足的关系概括为

$$\Pr_{H_0}\left(p_{exact}\le \alpha \mid Q\right)\le \alpha .\tag{4-1}$$

式（4-1）是分析所依赖的条件性命题，不是对任意时间序列都成立的保证。完整枚举只保证把给定交换空间数清楚；如果样本仅允许共同改变符号，却按相互独立的符号空间计算，得到的概率就不再对应实际数据模型。

条件上界可以直接由有限计数证明。固定任意成对差向量，再在全部标签组合上均匀赋标签。将统计量从大到小按等值分块；一个块的含等值上尾计数是截至该块的累计标签数。所有满足阈值的整块总大小至多为阈值与标签总数乘积的向下取整，因此有

$$\Pr\{p(S)\le \alpha \mid d\}\le \frac{\lfloor \alpha M \rfloor}{M}\le \alpha \tag{4-2}$$

其中，M 为全部标签数。十二对、4096 种标签及 0.01 阈值对应最多 40 个拒绝标签。并列或零差保留重数，可能使该界更保守；时间值可以来自相依序列，因为此命题的随机性仅来自固定时间之后的标签。一般变换群还需相应的分布不变性；零均值和边际对称都不够。[10]

本作品以共同符号模型检查这一边界。所有正幅度共享一个等概率符号时，各边际与联合全局中心对称均成立，实际随机状态却只有两个。盲用十二个独立符号的4096状态空间，会对两个实际结果均给出2/4096的尾概率，在既定阈值下每次拒绝；使用正确的全局翻转空间，尾概率为1。对本模型中明示共同符号依赖的输入，现有入口报告证据不足。程序只能核查部分声明和记录，不能自动证明真实数据符合交换条件；这不等于已经降低真实密码误报率。

对真实参考时间的后赋标签控制，则先固定时间值，再独立等概率地赋予交换标签。此时随机性来自标签，给定时间本身可以相关。控制结果只适用于这个采集完成后的标签模型；它不能证明原采集的密文类别具有同样随机化机制，也不用于推断真实密码总体的误报率。

样本数量还约束检验的分辨率。令k为非零成对差的数量；只要k为正，全局符号反转产生相同绝对和值，零差坐标又保留重复标签，因此任何观测状态的尾概率至少满足

$$p_{min} \geq 2^{1-k} \tag{4-3}$$

全零时尾概率为1。在0.01阈值下，七个或更少非零对不可能拒绝；八个非零对仅意味着有拒绝可能。允许4至20对的程序入口包含软件验证规模，不等于每个规模都具备检出能力。零差不能为通过而删除，更多单位也不自动保证真实噪声下的功效。

原秩检验的效应量也须按平均秩解释：第一类严格较大比例加上相等比例的一半。两类均为四个值1、1、2、2时，严格较大比例为0.25、相等比例为0.5，报告系数为0.5。旧字段名称继续保留，不能将它解释为严格大于概率。

4.5　复现一致性与研究结论的区分

复现一致性关注同一材料和规则能否得到相同输出。设归档文件为 D，固定分析规则为 F，原输出为 Y，则逐字复算核对的是

$$F(D)=Y. \tag{4-4}$$

这项等式成立，说明计算规则和文件关系能够被重建，却不自动证明 D 对目标总体有代表性，也不证明统计假设已满足。正因如此，系统同时保存方法说明、数据资格和适用范围，而不是只把“结果一致”作为全部验收的替代。

封存包重跑和同机复测已于2026年10月8日完成，分别支持材料重建与特定机制观察，旧ARM统计边界不变。截图与审批签署仍待真实材料，各项验收分别记录。

第 5 章　系统测试与实验结果分析

5.1　原生测试环境与分层验证方案

系统测试覆盖功能正确性、工程行为、统计实现和材料复算。各层测试的输入、预期结果及输出形式不同，因此分别报告：独立向量说明标准功能，接口与失败检查说明程序边界，统计夹具说明计算正确性，真实环境记录说明具体产物的执行情况。图 5.1 给出测试层次。



图 5.1　功能、软件、统计与材料的分层验证

表 5.1　原生 Linux 测试环境与构建条件

项目	配置或状态
操作系统	Ubuntu 24.04.4；Linux 6.8.0-138
执行架构	x86_64；KVM 来宾；4 vCPU
CPU 型号字符串	AMD EPYC 9K65 192-Core Processor
正式构建	GCC 13.3.0；RelWithDebInfo
正式编译条件	-O2 -g -DNDEBUG；无 LTO；非 sanitizer
动态检查	Clang 18.1.3；ASan / UBSan；另行运行
普通正式采集	外部 taskset 绑定 CPU2
计时口径	CLOCK_MONOTONIC；ns；完整解封装
环境限制	宿主频率、物理资源及调度不由来宾完全控制

CPU 型号中的“192-Core”是来宾报告的型号字符串，不能解释为本实例拥有 192 个核心。不同源码快照、编译器和实际执行窗口分别记录，不把 macOS/ARM 历史材料写成 Linux 实测，也不把动态检查耗时放入正式性能对照。

5.2　独立向量验收与工程边界检查

测试选用 25 组密钥生成、25 组封装、10 组解封装和 20 组密钥校验，共 80 组独立来源向量，在四种实现上累计完成 320 次验收。其中，解封装用例包含 5 组隐式拒绝输入。预期答案来自独立 NIST 向量，不由被测实现生成。接口测试进一步覆盖空指针和错误长度，三组对参考版的差分检查各覆盖 64 个公开种子及三类输入。

表 5.2　主要功能与软件检查结果

检查对象	覆盖范围	已有结果
独立已知答案	四种标签，每种 80 组	累计 320 次向量验收通过
无效密文语义	每种含 5 组隐式拒绝向量	按已执行用例通过
接口与等价	空指针、长度、三组差分	相应检查通过
随机源故障	open 失败传播	已测；read / close 注入未执行
N01 正式构建	GCC 普通 CTest	34 项通过
N01 动态检查	Clang ASan / UBSan CTest	34 项通过
实际产物	后端、include、值屏障、栈标记	固定配置核对通过

表中 320 次是 80 组独立来源向量在四种实现上的累计验收次数，34 项是当前构建注册的 CTest 项数，两者不是同一个计数单位。历史快照曾分别注册 31、32 或 33 项测试，报告按当时执行记录保留，不用后来的 34 项回填旧快照。此前 GCC 动态构建中的警告失败也保留为原始记录，后续 Clang 通过不能抹去该失败。

统计边界检查包括四个相同差值时的尾计数 2/16、零总和时的 16/16、全并列、整数范围、畸形输入和不适用资格。它们直接针对计数定义和失败路径，避免只检查少数正常样例。四个相同非零差值对应的概率为 0.125，在阈值 0.01 下不拒绝，这也表明系统保留有限分母产生的离散性。

5.3　普通解封装性能与跨运行波动

普通性能采用六个运行块。每块 reference、hardened、deployable 各执行 3000 次正式调用和 100 次预热，三种实现共享该块的公开输入流及类别比例。六种方法排列

全部出现，用以记录不同运行顺序下的结果。一次完整采集形成 18 份运行记录，共 54000 条正式时间与 1800 次预热。

图 5.2 展示旧快照 E02 的全部运行均值。三种实现的均值均处于约 46 μs 附近，运行之间存在波动，deployable 在第 4 块的均值相对较高。由于图中纵轴从 45 μs 起始，视觉上放大了小幅变化，应结合表 5.3 的原始数值阅读。

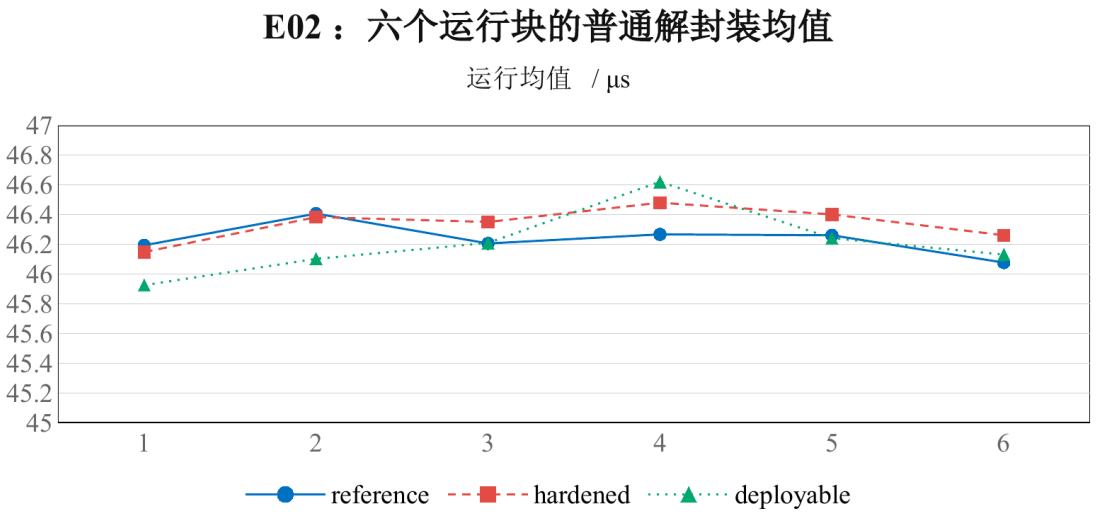


图 5.2　E02 六个运行块的普通解封装均值；纵轴为非零起点

表 5.3　E02 的逐运行均值（单位：ns）

运行块	reference	hardened	deployable
1	46193.517	46148.423	45925.623
2	46407.084	46383.034	46102.585
3	46206.193	46350.749	46209.969
4	46267.377	46480.338	46621.075
5	46261.240	46401.231	46241.867
6	46077.670	46261.061	46131.357

来源：E02 原始 C 性能报告；均值保留至 0.001 ns 用于对应原表，不表示时钟具有该精度。

表 5.4　E02 的平均差、相对开销与探索性区间

比较对象	平均差 / ns	相对开销	99%探索区间 / ns
hardened − reference	101.959	0.220522%	[−3.725, 189.399]
deployable − reference	−30.101	−0.065103%	[−240.918, 193.997]

两个区间均跨零。区间按运行对独立同分布的 bootstrap 模型生成，而 KVM 环境下这一条件未被证明，故只作探索描述。hardened 保留函数级 O0 控制，reference 与 deployable 正常优化，比较同时涉及优化条件与代码形态，不能把全部差异都归因于防护措施。

新快照 N01 另行执行同规模普通采集，结果如图 5.3 所示。其第 1 块 reference、第 2 块 deployable、第 5 块 hardened 和第 6 块 deployable 均出现较高运行均值，说明有限运行内仍有环境波动。新旧两组记录有各自源码和产物身份，本文并列报告，不将它们合并为同一总体估计。

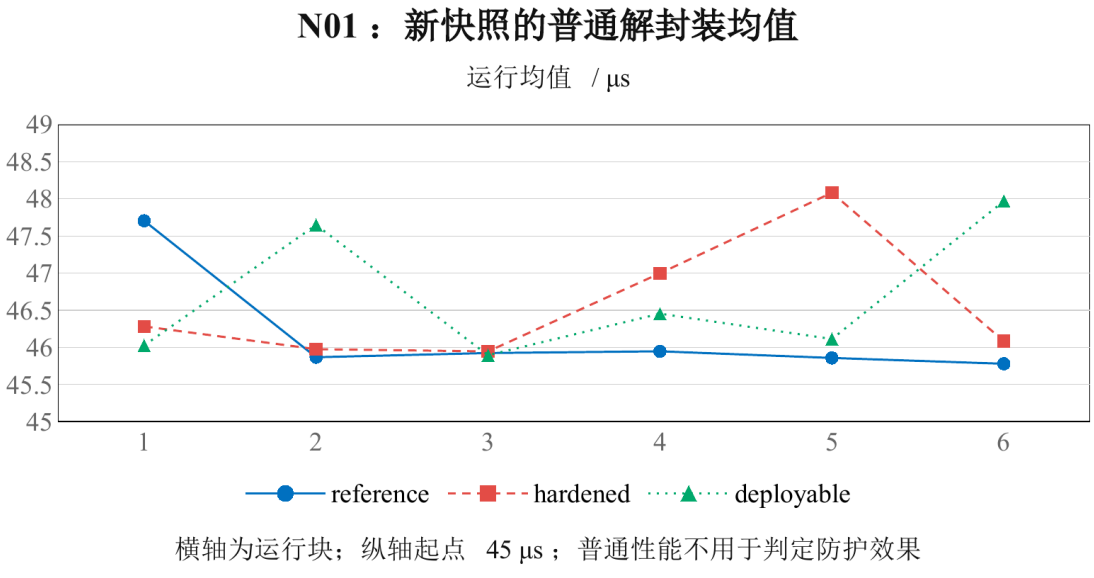


图 5.3　N01 新快照的六个运行块均值；纵轴为非零起点

表 5.5　N01 普通性能的比较结果

比较对象	平均差 / ns	相对开销	99%探索区间 / ns
hardened − reference	380.802	0.824599%	[−769.377, 1518.392]
deployable − reference	504.065	1.091517%	[−806.367, 1730.791]

N01 两个探索区间同样跨零。完整父窗口合计约 4.162 s，正式记录 54000 条、预热 1800 次。结合 E02 与 N01，可以描述普通负载下三种产物的运行开销及波动，这些普通负载测量本身不用于判定防护有效；同机人工机制的前后结果见第 5.8 节。

5.4　精确复算效率与成本边界复核

效率比较限定为无秘密成对数据的精确统计复算。强基线采用 SciPy 1.16.3 / NumPy 2.3.5，使用完整 2^{20} 种交换、相同整数差向量、相同绝对和值统计量及包含等值的上尾。基线启用向量化与固定批量，候选使用 C11 整数枚举。两者首先核对尾计数和判定一致，再比较耗时，避免以不同工作量形成表面优势。

开发阶段使用两份固定文件、12 次进程进行兼容检查与选择；确认阶段采用八份新冻结任务，每方法每任务重复三次，共 48 次完整进程。所有尾计数与判定一致，每个任务的最大 C 耗时均小于该任务的最小 SciPy 耗时。设第 q 个对应重复的候选与基线窗口分别为 C_q、B_q，比较比值定义为

$$\rho_q = \frac{C_q}{B_q}. \tag{5-1}$$

原确认中 24 个对应比值的中位数为 0.001388504，范围为 0.001139408 至 0.001734854。这个中位数是逐次比值的汇总，不等同于总耗时之比。确认观察窗口为 56.487808 s，开发为 14.048571 s，均按原冻结预算记录。

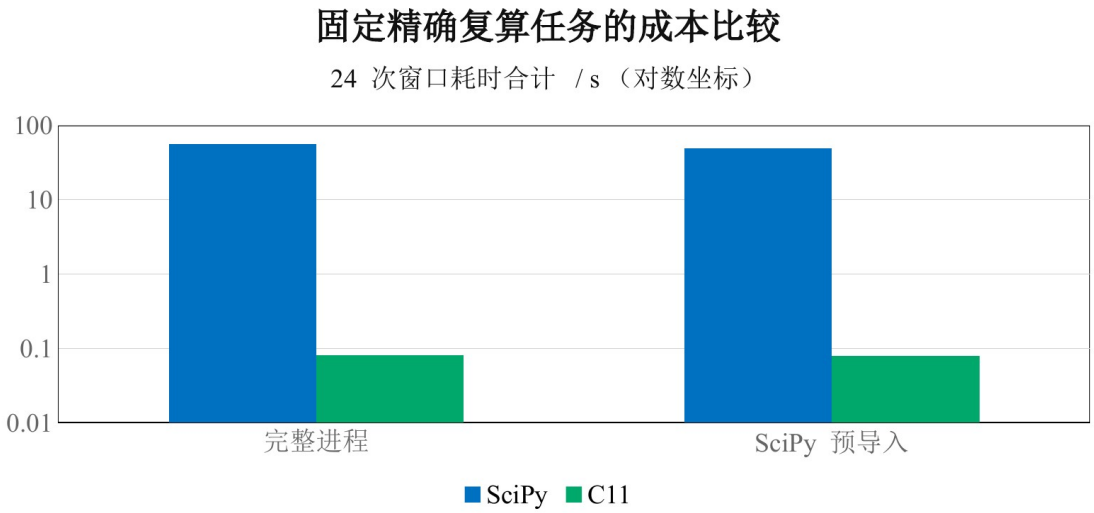


图 5.4　两种成本边界下的精确复算耗时；纵轴为对数刻度

表 5.6　固定精确复算的成本与一致性

项目	完整进程确认	SciPy 预导入复核
固定任务数	8	同一 8 个任务
两方法进程 / 窗口总数	48	48
SciPy 窗口合计 / s	56.401718	49.704188
C 完整进程合计 / s	0.080189	0.079016
SciPy 就绪成本 / s	计入完整进程	6.559904，另列
逐次比值中位数	0.001388504	0.001562070
精确尾计数及判定	全部一致	全部一致

为检查启动与导入成本是否决定结论，补充复核允许 SciPy 预先导入，候选仍承担完整进程成本。该窗口下的比值中位数为 0.001562070，任务内耗时分离关系仍保持。补充使用的是既有八个任务，属于成本边界敏感性检查，不能作为新的独立确认样本与原结果合并。

这些结果支持固定规模精确复算的专用效率增量，能够减少同一统计规则被反复核查时的等待时间。它们不表示 ML-KEM 解封装更快，也不表示检验更灵敏或防护更有效；没有据此给出任务总体的速度区间，研发完整墙钟亦未作为已采集指标。

为检查增量实现本身的成本收益，同语言比较复用既有八个任务，不增加独立确认任务。两入口保留相同输入校验、Gray 状态顺序、绝对和值、含等值上尾及结果整

理；完整求和入口在每个状态重算 20 项，增量入口更新改变的一项。两者均以 Apple Clang 21、C11 和-O2 独立编译，关闭 LTO，允许自然向量化。机器码保留了增量定位与单项更新，以及完整求和的 NEON 和标量计算。本次在 Apple M5、macOS 26.6.2/ARM64 的同一构建和进程内执行，未设置频率控制，也未记录绑核或后台负载隔离。

每方法每任务预热一次，再执行十对正式调用，每对方法先后次序交替，共保存 16 次预热、160 个正式窗口及 176 次有效结果消费。计时覆盖 compute_interval=0 的完整统计函数，包括校验、初始化、枚举和结果整理；读取、预热、结果核对、消费及 I/O 位于窗口外。八任务只有七种不同 CSV；任务 3 与 7 为字节相同的全零文件，保留但不用于逐状态更新收益主张。六份非零输入的十对增量/完整求和比均小于 1，各任务中位比为 0.261640 至 0.319174，八任务完整读数见表 A.2。每次尾计数均与归档独立 SciPy 期待一致；独立 C 审计进一步核对原始行、比值、摘要及输入副本。

CLOCK_MONOTONIC 报告分辨率为 1000 ns。预定门槛取时钟分辨率与空窗口中位值较大者的 100 倍，本次为 100000 ns，80 对均满足；保存 512 个校准原值，不扣读钟开销。正式函数窗口累计 0.375106 s，C 管理窗口另列为 0.426083 s，包含准备、读取、校准、预热、计算、核对、记录、复查及原输出关闭，不含最终状态发布、外部摘要和进程尾部。30 s 预算在调用及文件操作边界检查，未耗尽，不提供硬实时抢占保证。本结果描述固定输入的完整统计函数成本；差异还含加载、符号选择和改变位定位，不能解释为纯加法次数比，不能定量分解旧 C/SciPy 总比或推出总体速度、密码运算加速及检测能力改善。

5.5　有限模型下的误拒绝与检出表现

为独立验证统计实现，作品在观察结果前固定 12 个成对单位及有限模型。设效应为 δ，幅度为 a_k，独立等概率符号为 $\xi_k \in \{-1,+1\}$，差值定义为

$$d_k = \delta + \xi_k a_k . \tag{5-2}$$

其中，equal 模型各对幅度为 1，graded 模型各对幅度依次为 1 至 12；效应取 0、1、2、4、12。每个独立模型均枚举全部 4096 种符号组合。因此，模型内拒绝概率可以由有限计数直接得到：

$$\pi_{model}(\delta) = \frac{\#\{\xi : p_{exact}(d) \le 0.01\}}{4096} . \tag{5-3}$$

全部独立模型及共同符号边界模型共形成 40964 份数据。数据由 C 生成与验证，单位为合成 ns，不是新采集的密码执行时间；新的解封装调用数为 0。现有精确入口与另行实现的整数分布卷积逐份核对，尾计数全部一致，44 份预定 SciPy 审计亦一致。

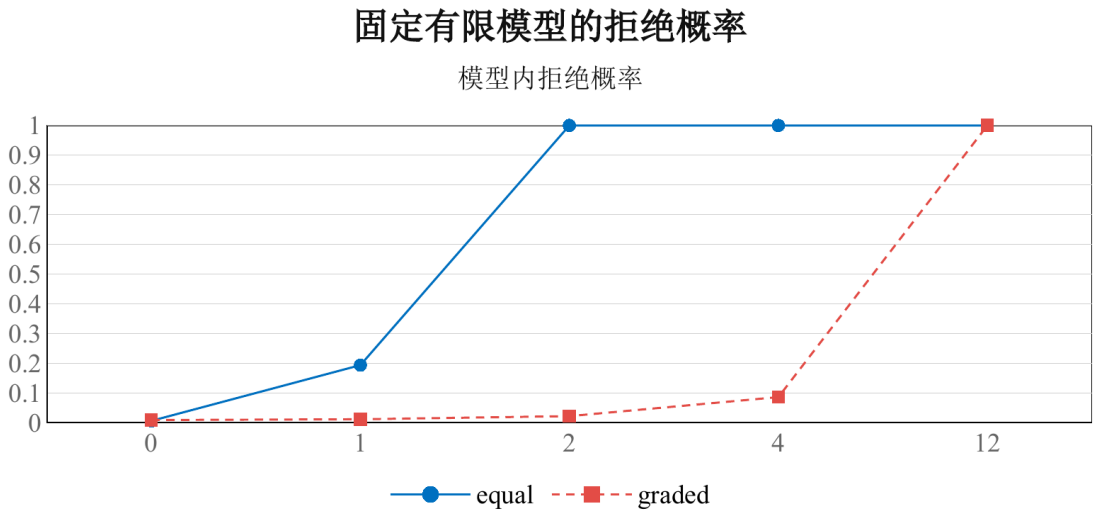


图 5.5　固定有限模型的拒绝概率；横轴为等距效应类别

表 5.7　独立模型的完整拒绝计数（每项分母 4096）

效应 / 合成 ns	equal 拒绝数	graded 拒绝数
0	26	38
1	794	48
2	4096	91
4	4096	353
12	4096	4096

零效应时，两模型的误拒绝概率分别为 26/4096 和 38/4096，均低于 0.01。非零效应下，equal 较快达到完整检出，而 graded 在效应 4 时仅 353/4096，说明检出程度受幅度结构影响，不能只以有利模型概括性能。equal 在效应 1 时出现的一份全零数据保留在分母中。

共同符号模型则每个仅有两种可能结果。若仍盲用独立交换空间，两个结果均会拒绝，两个模型分别出现 2/2；实际入口对四份依赖数据均返回证据不足。该结果说明资格判断的重要性，但没有据此证明真实密码时间序列已被校准，也没有构造与正确配置 dudect 或 RTLF 的检测能力胜负关系。

5.6　真实参考时间的后赋标签控制

后赋标签控制使用 E02 的六份封存 reference 记录。程序先完整读取每份 3000 条数据以及元数据、成本记录，并核验身份和完成状态；再固定取原序 0 至 23 共 24 个时间值，按相邻顺序组成 12 对。时间值不增加人工偏移，也不按原密文类别重新分组。

对给定时间对，采集后独立等概率赋标签，枚举全部 4096 种组合。记第 b 块固定时间向量为t_b，拒绝标签数为R_b，则条件概率为

$$\pi_b = Pr_{labels}(reject \mid t_b) = \frac{R_b}{4096}. \tag{5-4}$$

这一控制固定了时间值，把概率空间明确限定为标签。它允许给定时间之间存在相依，却不表示原采集过程已经实施同样的随机化。程序保存 144 个原时间及 24576 份差值和计数，新的解封装调用为 0。

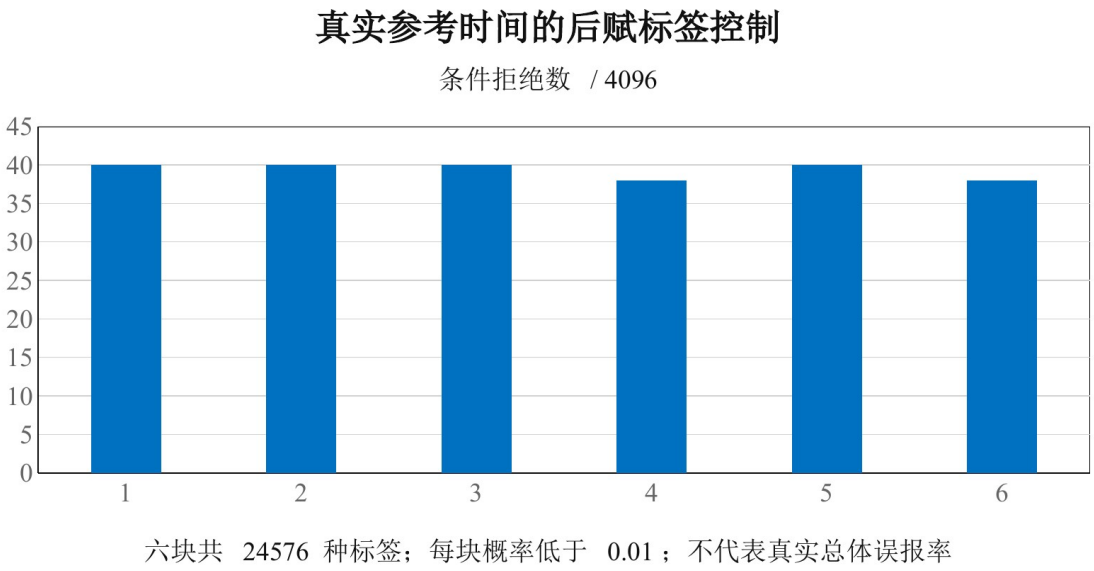


图 5.6　六个参考运行块的后赋标签条件拒绝数

表 5.8　后赋标签控制的全部结果

运行块	标签组合数	条件拒绝数	条件概率
1	4096	40	0.009765625000
2	4096	40	0.009765625000
3	4096	40	0.009765625000
4	4096	38	0.009277343750
5	4096	40	0.009765625000
6	4096	38	0.009277343750

六块条件概率均不超过 0.01。独立逐项求和与排序秩核对、24 份预定 SciPy 审计及完整动态运行均与原始结果一致。由于标签空间被完整枚举，这里的概率没有 Monte Carlo 抽样误差；但它仍只针对给定时间下的标签模型，不能解释成真实密码总体误报率估计。

初版曾遗漏成本内容资格校验，保留为未签收记录；修复后复用共同 C 读取器，使时钟、身份、计数及成本矛盾在输出控制数据前被拒绝。最终 C 任务耗时约 0.209 s，读取、计算和原始记录关闭均计入，最后摘要及状态发布另列。该修正提高了材料校验的完整性，没有改变原始时间或统计规则。

5.7　历史案例审计与封存材料复算

历史人工档案覆盖五个种子与四种实现标签，共 20 份数据生成 40 份报告。它们按照原分析器重建，并与归档结果逐字一致；本次报告重写未重新采集这些数据。表 5.9 列出对结论有直接影响的历史观察。

表 5.9　历史人工案例的观察及限制

项目	原有观察	解释限制
H1 主比较	5 个种子中 4 个达到 10^{-4}	未在全部种子稳定显著
H2 人工回归	5/5 达到 10^{-3}	名义概率未修正序列依赖
H2 reference / deployable	各 5/5 未检出	未检出不等于恒时证明
H2 hardened	1/5 仍有显著残差	不能声称差异全部消除
固定消息组	少量固定密文重复测量	调用数不是独立输入数

有限批次路线在第一次原生准备时因 GCC 结构体初始化编译错误停止。K4/K16 开发、选择和 20 对确认未执行，40000 次共同调用及 3600 秒预留未闭合。后续可移植性修复和普通验收不回填为该合同成功，也没有从停止记录产生稳定性收益或成本比值。

2026 年 10 月 8 日的封存 v3 重跑，则属于已经完成的工程复算。包摘要与本地封存文件一致，206 项源清单核对通过，26 个执行步骤全部退出 0；GCC 普通及 Clang 动态检查各 34 项通过。普通性能主表、精确主表、40964 份模型原始结果、24576 份标签结果以及 144 个原时间均逐字节一致，历史报告重分析成功。60 条普通短演示只说明入口可运行，不作统计结论。

5.8　同机人工机制的防护前后复测

新一轮复测在同一 Ubuntu 24.04.4、x86_64、KVM、4 vCPU 环境执行，匿名源码标识为 SOURCE-KVM-R07，正式产物采用 GCC 13.3.0 和 RelWithDebInfo，关闭 sanitizer。功能检查与计时使用同一构建，37 项检查全部通过。这一构建额外包含测量相关检查，与此前 34 项默认交付构建分别记录，不改写旧检查数量。

计划在采集前保存，五个预定种子全部执行，四种实现标签各形成五份正式记录。每次运行保存 25000 条正式时间及 100 次预热，63 个执行步骤全部退出 0。外部命令记录显示绑定 CPU2；元数据中的亲和性字段未读取外部设置，仍为 not_pinned，因此报告依据实际命令说明亲和性，而不改写原字段。宿主频率未配置，时钟的 1 ns 标称分辨率也不解释为 1 ns 实际精度。

本轮使用历史分析路径中的名义秩检验：十项比较的单项阈值为 10^{-4}，分组比较阈值为 10^{-3}，均不同于第 2 章精确随机化路径的 0.01。人工分支形态在五个种子的十项比较中均有 7 项达到原阈值，预定主比较均显著；reference、hardened、deployable 的主比较及十项显著数均为 0。“主比较显著”指原秩检验判定，不表示所有其他统计量都显著。

秩差异与均值差异在本次记录中并不总是一致。人工分支的种子 20261111，主比较 P50 差为 −19 ns、均值差为+42.340 ns，秩 p 为 5.46693e−7，而辅助 Welch p 为 0.350711。两者均是现有分析器的名义正态近似；辅助路径未按所报自由度计算 Student-t 尾概率。该例用于解释统计对象，不构成新的确认集合，长尾及全部种子均保留。

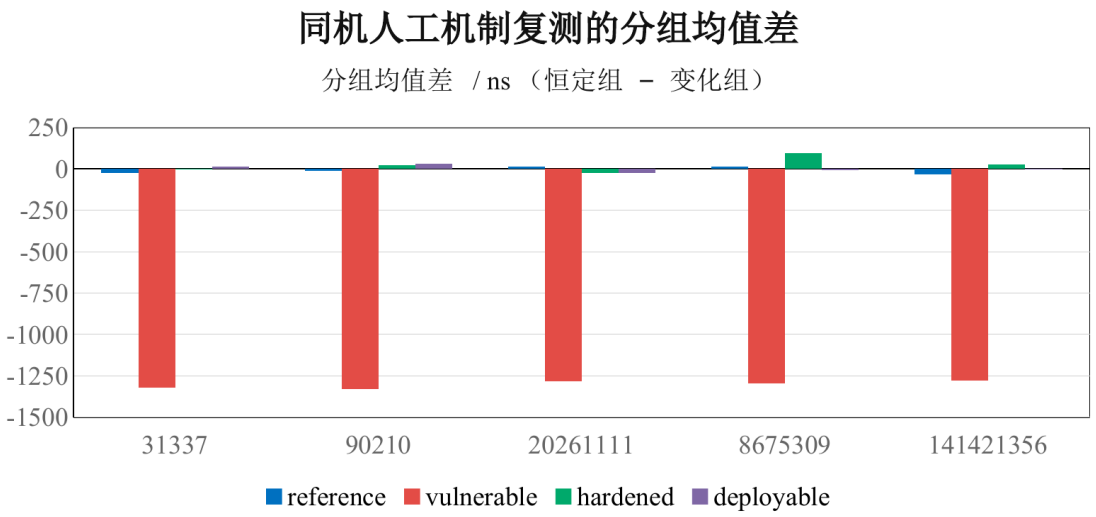


图 5.7　同机人工机制复测的分组均值差；方向为恒定组减变化组

表 5.10　同机复测的分组均值差（单位：ns）

种子	reference	vulnerable	hardened	deployable
31337	−23.760	−1322.119	0.693	15.051
90210	−12.891	−1329.081	20.532	29.601
20261111	15.184	−1283.025	−26.042	−25.086
8675309	13.212	−1296.339	95.723	−8.106
141421356	−31.964	−1277.020	27.345	−3.191

来源：R07 各实现、各种子的原始分组报告。表中是分组均值差，显著性另按原名义秩检验判定；不以差值本身替代统计判断，也不绘制未经计算的误差棒。

分组比较中，vulnerable 五次均显著，均值差约−1277 至−1329 ns；hardened 五次均未显著，差值约−26 至+96 ns，reference 与 deployable 也均未显著。该结果说明，在本次虚拟机及既定检验下，人工机制改为掩码形态后未再检出相应差异。它不证明差值严格为零，也不证明全库恒时；原报告中近似概率输出的零值不解释为数学上的精确零。

恒定消息两类仍是重复密文，名义检验对序列依赖的限制继续存在。上游目标函数原本无分支，本轮是已有人工机制的防护对照，不是新上游漏洞修复。旧 Apple M5 档案中的主比较失败和掩码残差仍保留，与本轮 Linux 记录分开解释。最新验收表据这些实际材料将 R07 记为范围内签收，而非全库安全认证。

第 6 章　作品特色与应用前景

6.1　面向固定任务的专用复算效率

本作品的实证增量首先体现在固定规模成对统计的复算效率。在保持数据、交换空间及尾部计数定义不变的条件下，C11 专用实现减少了固定任务的计算与运行开销经过正确配置的 SciPy 强基线比较，以及预导入成本敏感性检查，八个既定任务上的结果一致性和成本差异均得到验证。同语言对照为增量实现的完整函数成本提供直接观察；旧 SciPy 对照继续描述流程成本，两者不作定量分解。

这项增量服务于统计审查的重复核对环节。使用者需要更换机器、检查材料或重建报告时，可以沿用同一精确规则快速获得结果，而不必简化统计定义以缩短等待。其价值在于固定任务的工程效率；成对随机化、完整枚举和相邻状态更新均有既有方法基础，因此不把实现优化表述为全新统计理论。

这一改进对应题面统计分析流程的工程效率：比较前后使用相同统计量、完整交换空间和含等值尾规则，检出判定保持一致，减少的是复核等待和计算成本。正确性先于速度核对；八个既定任务之外的总体速度及检测能力不由这些结果保证。

6.2　数据资格与解释范围的一体化设计

作品将材料完整性、分析资格和统计数值分别处理，再在输出中关联呈现。这样，概率很小但交换条件不足的输入，不会仅凭数值被包装为完整确认；文件缺少成本或身份存在冲突时，也不会因能够读取部分行而被判为完整运行。

有限模型与后赋标签控制为这一设计提供了可复核例子。前者保留低检出、全零及依赖模型失配的结果，后者明确区分固定时间与随机标签。使用者因而能够看见结论从何而来、在哪个范围成立，以及哪些问题尚未解决。程序并不替代实验设计判断，而是减少材料和输出之间的隐含前提。

表 6.1　作品特色及其验证依据

特色	验证依据	适用范围
专用整数精确复算	八任务对照及预导入复核	固定任务的流程效率
数据资格与数值分离	缺失、矛盾及依赖数据检查	工程输入约束与解释边界
有限模型可完整核对	40964 份计数与独立核对	固定有限概率模型
真实记录条件化控制	六块 24576 种后赋标签	给定时间的独立标签模型
材料与报告可重建	封存包新目录授权复算	固定版本、材料及执行记录

6.3　本地审计与密码工程教学

在密码工程开发中，本作品可用于先验收功能与接口，再保存普通运行记录，最后按材料条件选择描述或统计分析。开发者能够借此检查修改是否改变输出语义、报告是否绑定正确产物，以及性能差异是否受到构建条件影响。由于原始记录、成本和报告能够相互对应，复核过程不必从零整理散落文件。

在密码实现教学中，系统适合展示“算法正确”与“实现性质”之间的区别。学生可以阅读已归档的历史案例，比较源码意图、实际产物和统计输出，理解未检出与安全证明之间的距离。无秘密有限模型则提供清楚的概率空间，使适用条件、离散阈值和全零边界能够通过完整计数得到说明。

上述应用均以本地环境、固定 ML-KEM-768 便携实现及已说明的数据范围为前提。迁移至其他处理器、优化条件或后端时，需要重新核对产物和运行证据；应用到其他统计任务时，也需要重新检查交换依据与样本单位，不能仅复用本报告中的比值或阈值说明。

第 7 章　总结与展望

7.1　工作总结

本作品围绕 ML-KEM-768 的实现审查需求，完成 C11 本地审计与精确复算系统。工程通过固定上游快照、隔离构建和独立向量核对建立功能基础；通过完整解封装计时、逐次记录及来源关联保存运行证据；通过数据资格检查与限定精确统计，使数值结果能够在明确条件下解释。

原生 GCC 普通及 Clang 动态检查各 34 项通过。普通探索区间在 KVM 的 E02/N01 跨零，在 Ryzen 裸金属不跨零。八个固定任务与 SciPy 的尾计数及判定一致，预导入复核仍见流程成本差异。macOS/ARM 同语言对照另支持六份非零输入的完整函数成本收益，限本次构建和进程。40964 份有限模型与 24576 份后赋标签结果分别核对计算正确性和条件边界。封存软件 v3 已完成授权代理新目录复算。同机 Linux 人工机制的五种子复测中，掩码形态在原名义检验里均未再检出；局限与历史记录保留。

这些成果共同支持可运行、可检查和可重建的本地审计流程。历史人工案例的局限、未执行的有限批次及不利结果均予以保留，作品不以复算更快代替密码防护更强，也不以检查通过代替全部研究要求已完成。

7.2　现有限制与后续工作

后续工作需要在已有同机防护复测基础上，进一步解释重复密文、序列相依以及 KVM 调度对名义检验的影响。当前五个种子的掩码形态均未再检出，支持该人工机制在固定环境和原检验下的改善观察；跨环境稳定性、完整因果分离以及全库恒时性质仍不能由此推出。普通性能与防护效果继续按各自的数据和方法解释。

其次，固定任务的效率优势尚不能外推至任意数据规模或所有统计场景。当前成对单位上限为 20，超过该范围需要重新选择计算策略和验证边界；有限模型也不是实际密码时间噪声的完整描述。扩展工作应继续保留失败、低检出及不利结果，而不是仅选择能够强化结论的条件。

研究实质等价需事先规定估计对象、容许界和有效区间。未检出、低自相关或条件标签枚举不能证明差异足够小；依赖与离散方法仍须核实适用条件。

实际界面截图、正式题名与审批签署仍需团队完成。本文沿用已提供编号和暂定题名，后续按真实材料更新，各项验收分别记录。

参考文献

[1] NIST. Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203, 2024. https://csrc.nist.gov/pubs/fips/203/final . 潜在勘误适用性未核实。

[2] pq-code-package. mlkem-native: Formal Verification — Scope, Assumptions, Risks. 固定提交 b3ba7b32773e657dd37f6f87bce82528459ad8a4. https://github.com/pq-code-package/mlkem-native/blob/b3ba7b32773e657dd37f6f87bce82528459ad8a4/SOUNDNESS.md .

[3] Reparaz O, Balasch J, Verbauwhede I. Dude, is my code constant time? DATE, 2017. 工具固定提交 dc269651fb2567e46755cfb2a13d3875592968b5. https://github.com/oreparaz/dudect .

[4] Dunsche M, et al. With Great Power Come Great Side Channels: Statistical Timing Side-Channel Analyses with Bounded Type-1 Errors. USENIX Security, 2024. https://www.usenix.org/conference/usenixsecurity24/presentation/dunsche . 正式勘误：https://www.usenix.org/system/files/sec24_errata_dunsche.pdf .

[5] SciPy. scipy.stats.permutation_test. API 文档 1.16.2，实验环境 1.16.3/NumPy2.3.5. https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.permutation_test.html .

[6] clangover 项目. 历史 Kyber 编译行为资料. 固定提交 7f4d5dc162b77c362a34a0d52949f7a3e1b16d81. https://github.com/antoonpurnal/clangover .

[7] Dunsche M, et al. SILENT: A New Lens on Statistics in Software Timing Side Channels. arXiv:2504.19821v1, 2025. https://arxiv.org/abs/2504.19821 .

[8] Brumley B B. SoK: The Constant Time Model. WOOT, 2026. https://www.usenix.org/conference/woot26/presentation/brumley .

[9] Diaconis P, Holmes S. Gray codes for randomization procedures. Statistics and Computing, 1994, 4:287–302. DOI:10.1007/BF00156752.

[10] Hemerik J, Goeman J. Exact testing with random permutations. TEST, 2018, 27:811–825. DOI:10.1007/s11749-017-0571-1.

[11] Schneider M, et al. Breaking Bad: How Compilers Break Constant-Time Implementations. ASIA CCS, 2025:1690–1706. DOI:10.1145/3708821.3733909.

附录 A　实验材料与结论对应关系

正文中的实验数字均对应已保存的原始记录或 C 输出。下表提供材料索引，帮助复核人员区分平台、对象及结果类型。目录以项目根目录为起点，完整文件清单及摘要随相应证据保存；内部原件与匿名分发副本按各自清单解释，不混用摘要。

表 A.1　主要材料索引

编号	材料与位置	本文使用范围
E01	docs/evidence/native-reference-20261007	前置普通参考与读钟检查
E02	docs/evidence/native-final-20261007/performance	图 5.2；表 5.3、5.4；附录 B
N01	docs/evidence/candidate-round-20261007/native	图 5.3；表 5.5；原生检查
H01	docs/evidence/statistics-multiseed	表 5.9；历史案例限制
C01	docs/evidence/controlled-comparison-20261007	图 5.4；表 5.6 完整进程
C02	docs/evidence/finite-calibration-20261007	图 5.5；表 5.7
C03	docs/evidence/warm-replay-20261007	图 5.4；表 5.6 预导入
C04	docs/evidence/reference-label-control-20261007	图 5.6；表 5.8
R07	docs/evidence/r07-linux-20261008	图 5.7；表 5.10；同机人工机制复测
V3-R	docs/evidence/author-proxy-v3-20261008	封存 v3 授权代理重跑

V3-R 为本报告的材料索引简称，不改写原始记录身份。封存包 SHA-256 为 58d6a60460952cec2a9c1cbc478f35a36e904ecab44252fc594fb98df34f5d26，源清单 SHA-256 为 601a1020f30e17fc6a072d35f24f43e33acf87153f8cef8bf1c70475dad0a938。

裸金属记录单独列为 docs/evidence/baremetal-ryzen-20261008，不并入 docs/evidence/r07-linux-20261008。该环境是 Ubuntu 24.04.4，处理器为 AMD Ryzen 9 9950X。调速器保持 powersave，频率没有锁定。preflight.log 与 frequency-after.log 均为 governor=powersave，并记录 frequency_policy=unchanged_powersave、cur_khz=2982259、max_khz=5756452。

普通性能与机制 meta 写明 frequency_control=not_configured，cpu_affinity=not_pinned，clock=CLOCK_MONOTONIC，time_unit=ns，单位 ns。普通

性能为 18×3000 条正式记录，机制计时为 20×25000 条。collect/data.csv 为 60 条，calibrate/raw.csv 为 40964 条，reference-label-control/raw.csv 为 24576 条。这次运行没有另写 groups 报告。

performance.report 写明 comparison=hardened:reference mean_difference_ns=147.350555556 relative_overhead=0.004913136 interval_lower_ns=53.755111111 interval_upper_ns=272.396055556。

performance.report 写明 comparison=deployable:reference mean_difference_ns=66.029277778 relative_overhead=0.002201626 interval_lower_ns=22.683222222 interval_upper_ns=103.365777778。

同份报告写明 formal_completed_calls=54000，warmup_completed_calls=1800，interval_scope=exploratory_iid_run_pair_bootstrap_only，report_status=complete，anonymous_source_id=SOURCE-RYZEN-01。

机制 analyze 报告中，vulnerable 的 significant_comparison_count 均为 7，primary 比较 result=significant；reference、hardened 与 deployable 的 significant_comparison_count 均为 0，primary 比较 result=not_significant。

这里的二十次机制运行按变体分段执行，存在时间状态混杂；普通性能六块则覆盖六种方法排列，不能统称固定顺序。后赋标签命令实际读取历史 native-final-20261007 的 KVM 时间，而非新裸金属时间。这些输入来源与执行机器分别记录，不将重放结果解释为新平台校准。

sanitize-build 退出码为 2。GCC 在 -Werror=sign-conversion 处停止，函数为 baseline_native_poly_frommsg，位置是 vulnerable compress.c:709 与 hardened compress.c:710。这些记录不是频率锁定、恒时证明或竞赛签收。R02 与 R04 仍为部分签收：没有锁定频率的数据集，也没有真实密文总体的误报率或检出率。

自有源码使用匿名来源别名，不是 Git 提交号；第三方固定版本保持不变。公开来源与固定版本适用性分别见 docs/upstream-security-review.md 与 docs/current-version-defense-review.md；最新验收状态见 docs/full-work-acceptance.md。本报告引用重写期间已归档的新 R07 记录，没有为绘图额外运行密码采集，也未改动原始数字。图源以独立 PowerPoint 文件交付，各数据图保留可编辑图表和工作簿，结构图保留原生图形与文字。

表 A.2　同语言完整统计函数的固定任务耗时比

任务	非零差数量	增量/完整求和中位比	十对比值最小—最大
1	20	0.265067	0.241923—0.275621
2	20	0.318405	0.309984—0.325525
3	0	0.330575	0.325333—0.338235
4	20	0.261640	0.257364—0.266460
5	20	0.298269	0.292138—0.308256
6	20	0.319174	0.313590—0.328080
7	0	0.331370	0.325712—0.335913
8	20	0.296426	0.291052—0.306987

中位数汇总每任务十个对应窗口比值，不是总耗时比；十对重复来自同一构建和进程，不提供总体置信区间。任务 3 与 7 为同一全零 CSV，仍列出全部读数，但不参与六份非零向量的逐状态更新收益主张。

本轮材料独立保存于 docs/evidence/aris-update-cost-20261008，数学实现位于 docs/research/aris-20261008/update-cost。匿名数据解释件新增 M01_UPDATE_COST 组，包含原始时间、读钟、对应比值、阶段终态、独立 C 核验及匿名资格摘要；24 份输入链接既有 C01，不增加独立数据。匿名源码标识 SOURCE-MATH-COST-01 不是 Git 提交号；实际源与产物按文件摘要关联。新的研究补充件不替换已封存软件 v3。

附录 B　普通性能的完整分位数

为避免正文只展示均值而掩盖尾部，表 B.1 保留 E02 全部 18 次运行的 P50、P90 和 P99。各行对应一次 3000 条正式记录的运行，不把各行分位数直接平均解释为合并分布的分位数。

表 B.1　E02 逐运行描述指标（单位：ns）

块	实现	均值	P50	P90	P99
1	reference	46193.517	45720	45949	54699
1	hardened	46148.423	45908	46058	50968
1	deployable	45925.623	45688	45829	50667
2	reference	46407.084	45538	48678	60098
2	hardened	46383.034	45849	46129	56419
2	deployable	46102.585	45578	45808	56459
3	reference	46206.193	45688	45929	55897
3	hardened	46350.749	45808	46029	56530
3	deployable	46209.969	45689	45909	56238
4	reference	46267.377	45729	45950	56348
4	hardened	46480.338	45929	46169	56799
4	deployable	46621.075	45779	47489	60619
5	reference	46261.240	45719	45950	56279
5	hardened	46401.231	45849	46089	56408
5	deployable	46241.867	45718	45938	56498
6	reference	46077.670	45540	45879	56389
6	hardened	46261.061	45719	45949	56549
6	deployable	46131.357	45579	45789	56479

来源：E02 原始 performance.report。表中全部普通长尾参与原指标计算，正文作单位换算的图与本表来自同一组记录。N01 属于另一个源码和产物身份，其逐运行全指标保存在附录 A 所列原报告中。